```
//Top level module (should not need to change except to uncomment ADC module)
`default_nettype none

module top_level(   input wire clk_100mhz,
                    input wire[15:0] sw,
                    input wire btnc, btnu, btnd, btnr, btnl,
                    input wire vauxp3,
                    input wire vauxn3,
                    input wire vn_in,
                    input wire vp_in,
                    output logic [15:0] led,
                    output logic aud_pwm,
                    output logic aud_sd
    );
    parameter SAMPLE_COUNT = 2082;//gets approximately (will generate audio at approx
48 kHz sample rate.

    logic [15:0] sample_counter;
    logic [11:0] adc_data;
    logic [11:0] sampled_adc_data;
    logic sample_trigger;
    logic adc_ready;
    logic enable;
    logic [7:0] recorder_data;
    logic [7:0] vol_out;
    logic [17:0] filter_out;
    logic pwm_val; //pwm signal (HI/LO)

    assign aud_sd = 1;
    assign led = sw; //just to look pretty
    assign sample_trigger = (sample_counter == SAMPLE_COUNT);


    always_ff @(posedge clk_100mhz)begin
        if (sample_counter == SAMPLE_COUNT)begin
            sample_counter <= 16'b0;
        end else begin
            sample_counter <= sample_counter + 16'b1;
        end
        if (sample_trigger) begin
            sampled_adc_data <= {~adc_data[11],adc_data[10:0]}; //convert to signed. i
ncoming data is offset binary
            //https://en.wikipedia.org/wiki/Offset_binary
        end
    end

    //ADC uncomment when activating!
    xadc_wiz_0 my_adc ( .dclk_in(clk_100mhz), .daddr_in(8'h13), //read from 0x13 for a
                        .vauxn3(vauxn3),.vauxp3(vauxp3),
                        .vp_in(1),.vn_in(1),
                        .di_in(16'b0),
                        .do_out(adc_data),.drdy_out(adc_ready),
                        .den_in(1), .dwe_in(0));


    recorder myrec( .clk_in(clk_100mhz),.rst_in(btnd),
                    .record_in(btnc),.ready_in(sample_trigger),
                    .filter_in(sw[0]),.echo_in(sw[1]),
                    .mic_in(sampled_adc_data[11:4]), .data_out(recorder_data));


    volume_control vc (.vol_in(sw[15:13]),
                       .signal_in(recorder_data), .signal_out(vol_out));
    pwm (.clk_in(clk_100mhz), .rst_in(btnd), .level_in({~vol_out[7],vol_out[6:0]}), .p
```

```
wm_out(pwm_val));
    assign aud_pwm = pwm_val?1'bZ:1'b0;

endmodule



//////////////////////////////////////////////////////////////////////////////////
//
// Record/playback
//
//////////////////////////////////////////////////////////////////////////////////


module recorder(
  input wire clk_in,               // 100MHz system clock
  input wire rst_in,                // 1 to reset to initial state
  input wire record_in,             // 0 for playback, 1 for record
  input wire ready_in,              // 1 when data is available
  input wire filter_in,             // 1 when using low-pass filter
  input wire echo_in,               // 1 when using echo filter
  input wire signed[7:0] mic_in,          // 8-bit PCM data from mic
  output logic signed [7:0] data_out      // 8-bit PCM data to headphone
);
    logic signed [7:0] data_to_bram;
    logic signed [7:0] data_from_bram_a;
    logic signed [7:0] data_from_bram_b;
    logic signed [7:0] data_from_bram_c;
    logic [15:0] addr_a;
    logic [15:0] addr_b;
    logic [15:0] addr_c;
    logic wea;

    logic [17:0] filter_output;
    logic [7:0] filter_input;


    // Three Block Rams to Generate the Echo
    blk_mem_gen_0(.addra(addr_a), .clka(clk_in), .dina(data_to_bram), .douta(data_from
_bram_a),
                  .ena(1), .wea(wea));

    blk_mem_gen_1(.addra(addr_b), .clka(clk_in), .dina(data_to_bram), .douta(data_from
_bram_b),
                  .ena(1), .wea(wea));

    blk_mem_gen_2(.addra(addr_c), .clka(clk_in), .dina(data_to_bram), .douta(data_from
_bram_c),
                  .ena(1), .wea(wea));

    //FIR FILTER
    fir31 myfir31(  .clk_in(clk_in), .rst_in(rst_in),
                    .ready_in(ready_in), .x_in(filter_input),
                    .y_out(filter_output));

    logic [2:0] sample_count;
    logic [15:0] max_address;

    logic signed [17:0] echo;
    assign echo = (data_from_bram_a) + (data_from_bram_b >> 1) + (data_from_bram_c >>
2);


    parameter MAX_LINE = 64_000; //Max memory
```

```
    enum {playback, record} state;

    always_ff @(posedge clk_in) begin
        if (rst_in) begin
            addr_a <= 0;
            sample_count <= 0;
            addr_b <= 0;
            addr_c <= 0;
            max_address <= 0;
            wea <= 0;
            state <= playback;
        end else if (state == playback) begin
        // Playback state
            if (echo_in && ready_in) begin
                if(sample_count == 0) begin
                   if (addr_a == max_address) begin
                        addr_a <= 0;
                   end else begin
                        addr_a <= addr_a + 1;
                   end
                   filter_input <= echo ;
                   data_out <= (filter_in) ? filter_output[17:10]: echo;

                   if(addr_a >= 12000) begin
                        addr_c <= addr_a - 12000;
                        addr_b <= addr_a - 6000;
                   end

                   sample_count <= sample_count + 1;
                end else begin
                    sample_count <= sample_count + 1;
                end


            end else if (ready_in) begin
                if(sample_count == 0) begin
                   if (addr_a == max_address - 1) begin
                        addr_a <= 0;
                   end else begin
                        addr_a <= addr_a + 1;
                   end
                   filter_input <= (filter_in) ? data_from_bram_a: data_from_bram_a;
                   data_out <= (filter_in) ? filter_output[17:10] : data_from_bram_a;

                   sample_count <= sample_count + 1;
                end else begin
                   sample_count <= sample_count + 1;
                end
            end
            if(record_in) begin

                addr_a <= 0;
                addr_b <= 0;
                addr_c <= 0;
                state <= record;
            end
        end else if (state == record) begin
        // Record
            if (ready_in) begin
                if (sample_count == 0) begin
                    if(addr_a < MAX_LINE) begin
                        wea <= 1;
                        data_to_bram <= (filter_in) ? filter_output[17:10] : mic_in;
                        addr_a <= addr_a + 1;
                        addr_b <= addr_b + 1;
```

```
                        addr_c <= addr_c + 1;
                        max_address <= addr_a;
                    end
                    sample_count <= sample_count + 1;
                end else begin
                    wea <= 0;
                    sample_count <= sample_count + 1;
                end
                filter_input <= mic_in;
                data_out <= (filter_in) ? filter_output[17:10] : mic_in;
            end
            if(!record_in) begin
                addr_a <= 0;
                addr_b <= 0;
                addr_c <= 0;
                wea <= 0;
                state <= playback;
            end
        end
    end
endmodule

//////////////////////////////////////////////////////////////////////////////////
//
// 31-tap FIR filter, 8-bit signed data, 10-bit signed coefficients.
// ready is asserted whenever there is a new sample on the X input,
// the Y output should also be sampled at the same time.  Assumes at
// least 32 clocks between ready assertions.  Note that since the
// coefficients have been scaled by 2**10, so has the output (it's
// expanded from 8 bits to 18 bits).  To get an 8-bit result from the
// filter just divide by 2**10, ie, use Y[17:10].
//
//////////////////////////////////////////////////////////////////////////////////

module fir31(
  input  wire clk_in,rst_in,ready_in,
  input wire signed [7:0] x_in,
  output logic signed [17:0] y_out
);
  // for now just pass data through
  logic signed [7:0] sample [31:0];  // 32 element array each 8 bits wide
  logic [4:0] offset; //pointer for the array! (5 bits because 32 elements in above ar
ray! Do not make larger)
  logic [4:0] index;
  logic signed [17:0] accumulator;
  logic signed [9:0] coeff;

  coeffs31 coeff1(.index_in(index), .coeff_out(coeff));

  always_ff @(posedge clk_in) begin
        if(rst_in) begin
            offset <= 0;
        end else begin
            if (ready_in) begin
                sample[offset] <= x_in;
                offset <= offset + 1;
                accumulator <= 0;
                index <= 0;
            end else begin
                if (index == 30) begin
                    y_out <= accumulator;
                end else begin
                    accumulator <= accumulator + coeff * sample[offset - index];
                    index <= index + 1;
                end
```

```
            end
        end
  end
endmodule


//////////////////////////////////////////////////////////////////////////////////
//
// Coefficients for a 31-tap low-pass FIR filter with Wn=.125 (eg, 3kHz for a
// 48kHz sample rate).  Since we're doing integer arithmetic, we've scaled
// the coefficients by 2**10
// Matlab command: round(fir1(30,.125)*1024)
//
//////////////////////////////////////////////////////////////////////////////////

module coeffs31(
  input  wire [4:0] index_in,
  output logic signed [9:0] coeff_out
);
  logic signed [9:0] coeff;
  assign coeff_out = coeff;
  // tools will turn this into a 31x10 ROM
  always_comb begin
    case (index_in)
      5'd0:  coeff = -10'sd1;
      5'd1:  coeff = -10'sd1;
      5'd2:  coeff = -10'sd3;
      5'd3:  coeff = -10'sd5;
      5'd4:  coeff = -10'sd6;
      5'd5:  coeff = -10'sd7;
      5'd6:  coeff = -10'sd5;
      5'd7:  coeff = 10'sd0;
      5'd8:  coeff = 10'sd10;
      5'd9:  coeff = 10'sd26;
      5'd10: coeff = 10'sd46;
      5'd11: coeff = 10'sd69;
      5'd12: coeff = 10'sd91;
      5'd13: coeff = 10'sd110;
      5'd14: coeff = 10'sd123;
      5'd15: coeff = 10'sd128;
      5'd16: coeff = 10'sd123;
      5'd17: coeff = 10'sd110;
      5'd18: coeff = 10'sd91;
      5'd19: coeff = 10'sd69;
      5'd20: coeff = 10'sd46;
      5'd21: coeff = 10'sd26;
      5'd22: coeff = 10'sd10;
      5'd23: coeff = 10'sd0;
      5'd24: coeff = -10'sd5;
      5'd25: coeff = -10'sd7;
      5'd26: coeff = -10'sd6;
      5'd27: coeff = -10'sd5;
      5'd28: coeff = -10'sd3;
      5'd29: coeff = -10'sd1;
      5'd30: coeff = -10'sd1;
      default: coeff = 10'hXXX;
    endcase
  end
endmodule

//Volume Control
module volume_control (input wire [2:0] vol_in, input wire signed [7:0] signal_in, out
put logic signed[7:0] signal_out);
    logic [2:0] shift;
    assign shift = 3'd7 - vol_in;
```

```
    assign signal_out = signal_in>>>shift;
endmodule

//PWM generator for audio generation!
module pwm (input wire clk_in, input wire rst_in, input wire [7:0] level_in, output lo
gic pwm_out);
    logic [7:0] count;
    assign pwm_out = count<level_in;
    always_ff @(posedge clk_in)begin
        if (rst_in)begin
            count <= 8'b0;
        end else begin
            count <= count+8'b1;
        end
    end
endmodule


//Sine Wave Generator
module sine_generator ( input wire clk_in, input wire rst_in, //clock and reset
                        input wire step_in, //trigger a phase step (rate at which you
run sine generator)
                        output logic [7:0] amp_out); //output phase
    parameter PHASE_INCR = 32'b1000_0000_0000_0000_0000_0000_0000_0000>>5; //1/64th of
 48 khz is 750 Hz
    logic [7:0] divider;
    logic [31:0] phase;
    logic [7:0] amp;
    assign amp_out = {~amp[7],amp[6:0]};
    sine_lut lut_1(.clk_in(clk_in), .phase_in(phase[31:26]), .amp_out(amp));

    always_ff @(posedge clk_in)begin
        if (rst_in)begin
            divider <= 8'b0;
            phase <= 32'b0;
        end else if (step_in)begin
            phase <= phase+PHASE_INCR;
        end
    end
endmodule

//6bit sine lookup, 8bit depth
module sine_lut(input wire [5:0] phase_in, input wire clk_in, output logic[7:0] amp_ou
t);
  always_ff @(posedge clk_in)begin
    case(phase_in)
      6'd0: amp_out<=8'd128;
      6'd1: amp_out<=8'd140;
      6'd2: amp_out<=8'd152;
      6'd3: amp_out<=8'd165;
      6'd4: amp_out<=8'd176;
      6'd5: amp_out<=8'd188;
      6'd6: amp_out<=8'd198;
      6'd7: amp_out<=8'd208;
      6'd8: amp_out<=8'd218;
      6'd9: amp_out<=8'd226;
      6'd10: amp_out<=8'd234;
      6'd11: amp_out<=8'd240;
      6'd12: amp_out<=8'd245;
      6'd13: amp_out<=8'd250;
      6'd14: amp_out<=8'd253;
      6'd15: amp_out<=8'd254;
      6'd16: amp_out<=8'd255;
      6'd17: amp_out<=8'd254;
      6'd18: amp_out<=8'd253;
```

```
      6'd19: amp_out<=8'd250;
      6'd20: amp_out<=8'd245;
      6'd21: amp_out<=8'd240;
      6'd22: amp_out<=8'd234;
      6'd23: amp_out<=8'd226;
      6'd24: amp_out<=8'd218;
      6'd25: amp_out<=8'd208;
      6'd26: amp_out<=8'd198;
      6'd27: amp_out<=8'd188;
      6'd28: amp_out<=8'd176;
      6'd29: amp_out<=8'd165;
      6'd30: amp_out<=8'd152;
      6'd31: amp_out<=8'd140;
      6'd32: amp_out<=8'd128;
      6'd33: amp_out<=8'd115;
      6'd34: amp_out<=8'd103;
      6'd35: amp_out<=8'd90;
      6'd36: amp_out<=8'd79;
      6'd37: amp_out<=8'd67;
      6'd38: amp_out<=8'd57;
      6'd39: amp_out<=8'd47;
      6'd40: amp_out<=8'd37;
      6'd41: amp_out<=8'd29;
      6'd42: amp_out<=8'd21;
      6'd43: amp_out<=8'd15;
      6'd44: amp_out<=8'd10;
      6'd45: amp_out<=8'd5;
      6'd46: amp_out<=8'd2;
      6'd47: amp_out<=8'd1;
      6'd48: amp_out<=8'd0;
      6'd49: amp_out<=8'd1;
      6'd50: amp_out<=8'd2;
      6'd51: amp_out<=8'd5;
      6'd52: amp_out<=8'd10;
      6'd53: amp_out<=8'd15;
      6'd54: amp_out<=8'd21;
      6'd55: amp_out<=8'd29;
      6'd56: amp_out<=8'd37;
      6'd57: amp_out<=8'd47;
      6'd58: amp_out<=8'd57;
      6'd59: amp_out<=8'd67;
      6'd60: amp_out<=8'd79;
      6'd61: amp_out<=8'd90;
      6'd62: amp_out<=8'd103;
      6'd63: amp_out<=8'd115;
    endcase
  end
endmodule
`default_nettype wire
```